INVERTIS UNIVERSITY BAREILLY

DEPARTMENT OF COMPUTER APPLICATIONS

PROGRAM CODE: **BCA_AI_05**

PROGRAM NAME: **BCA AI (PROJECT-BASED LEARNING)**

PROJECT ID: **DA-005**

PROJECT NAME: **Food Delivery Time and Order Analysis**

MENTOR NAME: **MR. VISHNU**

DEPARTMENT OF COMPUTER APPLICATIONS

COURSE NAME: BCA (AI)

Section - A

GROUP ID: **BCA_AI_05** SEMESTER: **I**

Sr.	Team Member	Student ID	Roll No.	Project Role & Contribution
1	**SUDHEER KUMAR**	BCAI2026069	2610206191	Technical Head (Complete Python Coding, Data Preprocessing, Imputation, Chart Generation & GitHub Deployment)
2	**VYOM SHARMA (T.L.)**	BCAI2026071	2610206218	Team Leader; Presentation Team (Problem Statement & Introduction)
3	**SUBRAT MISHRA**	BCAI2026082	2610206190	Presentation Team (KPI Dashboard & Summary Metrics)
4	**ASHISH KUMAR**	BCAI2026131	2610206046	Presentation Team (Delivery Time Distribution Chart)
5	**ADITY PRATAP SINGH**	BCAI2026136	2610206010	Presentation Team (Distance vs Duration Scatter Plot)
6	**FAZILAT KHAN**	BCAI2026202	2610206072	Presentation Team (Time of Day Volume & Peak Hours)
7	**SAMEER RAJ**	BCAI2026223	2610206161	Presentation Team (Traffic Impact & Box Plot Analysis)
8	**SHALINI KUMARI**	BCAI2026232	2610206173	Presentation Team (Business Recommendations)
9	**RAHUL KUMAR**	BCAI2026235	2610206147	Presentation Team (Tech Stack & Repository Overview)
10	**HIMANSHU KUMAR CHANDEL**	BCAI2026236	2610206079	Presentation Team (Conclusion & Q&A Coordination)

Team Leader: Vyom Sharma. **Technical Head:** Sudheer Kumar, who completed the entire technical implementation of the project individually.

INDEX

1. Introduction.....04

2. Problem Statement.....06

3. Objective of Project.....08

4. Methodology and Data Preprocessing.....10

5. Literature Review and References.....13

6. Explanation of Project (Working).....14

7. System Architecture and Workflow.....17

8. Program.....19

9. Result and Discussions.....23

10. Future Scope.....27

11. Conclusion.....29

12. References (IEEE Style).....29

1. Introduction:

1.1 Background of Modern Food Delivery Logistics

The food delivery industry has undergone a remarkable transformation during the last decade. What began as telephone-based ordering from neighbourhood restaurants has matured into a sophisticated, technology-driven ecosystem in which millions of orders are placed, prepared, dispatched, and delivered every day. In India, platforms such as Zomato and Swiggy have become household names, and their operational models have reshaped how urban and semi-urban consumers think about meals, convenience, and time. A customer who opens a mobile application, selects a dish, and confirms payment initiates a chain of coordinated activities that involve the restaurant kitchen, a delivery partner (courier), a routing and dispatch engine, a payment gateway, and a customer-support layer. Every link in this chain contributes to a single number that the customer cares about above almost everything else: the total time between placing the order and receiving it.

The operational model of platforms such as Zomato and Swiggy is, at its core, a three-sided marketplace. On one side are restaurants and cloud kitchens that supply food. On the second side are delivery partners who supply last-mile transportation capacity. On the third side are customers who generate demand. The platform sits in the middle and attempts to match these three sides in real time while optimizing cost, speed, and quality. Unlike traditional e-commerce, where a parcel may be delivered after one or two days, food is a highly perishable product. A burger that arrives cold, a pizza that arrives soggy, or a hot curry that has lost its temperature loses a significant portion of its value in the minutes after it leaves the kitchen. Consequently, the tolerance for delay is extremely small, and delivery duration becomes the most visible indicator of service quality.

The typical lifecycle of an order can be divided into several stages. First, the order is placed and confirmed by the restaurant. Second, the kitchen prepares the food, a stage described in the dataset used in this project through the variable *Preparation_Time_min*. Third, the dispatch engine assigns a courier, taking into account the courier's location, current load, vehicle type, and experience. Fourth, the courier travels to the restaurant, collects the order, and rides to the customer's address, a stage strongly influenced by distance, weather, traffic, and time of day. Finally, the order is handed over and the transaction is closed. Delays can be introduced at any stage, but the travelling stage is the most exposed to external and uncontrollable factors, which is why it is of particular analytical interest.

1.2 Importance of Delivery Duration and SLA Compliance

In logistics, a Service Level Agreement (SLA) is a formal or informal commitment regarding the quality, speed, or reliability of a service. In food delivery, the most commonly advertised SLA is the promised delivery time. Many platforms display an estimated time of arrival (ETA) on the checkout screen, and customers use this figure to decide whether to place the order at all. For the purposes of this project, a delivery-time threshold of **45 minutes** has been adopted as the benchmark SLA. Any order whose *Delivery_Time_min* exceeds 45 minutes is classified as a delayed order. The choice of 45 minutes is consistent with the popular industry expectation that a typical urban meal order should reach the customer within three-quarters of an hour, allowing for preparation, pickup, and transit.

SLA compliance matters for several reasons, which can be grouped into customer, business, and workforce perspectives.

Customer perspective. Delivery duration is one of the strongest drivers of customer satisfaction and retention. A customer who experiences repeated late deliveries is likely to uninstall the application, switch to a competitor, or post negative reviews that influence prospective customers. Because

switching costs in the food delivery sector are almost zero, a single poor experience can result in permanent loss of a customer.

Business perspective. Delayed orders generate direct and indirect costs. Direct costs include refunds, discount vouchers, and compensation offered to dissatisfied customers. Indirect costs include reduced repeat-order rates, brand damage, and increased load on customer-support teams. Restaurants are also affected, because late delivery degrades food quality and leads to complaints that are often wrongly attributed to the kitchen.

Workforce perspective. Delivery partners are under constant pressure to meet time targets. When the system sets unrealistic ETAs for long distances or congested periods, couriers may feel compelled to ride faster, take unsafe shortcuts, or work extended hours, which results in fatigue, accidents, and attrition. Accurate understanding of delay patterns therefore also contributes to the welfare and safety of the delivery workforce.

The present project observes that **66.4 percent** of the orders in the analysed dataset exceed the 45-minute threshold. This is a very high figure and signals that either the SLA is unrealistically strict for the conditions represented in the dataset, or that significant operational inefficiencies exist. Either interpretation justifies a thorough analytical study.

1.3 Role of Data Analytics in Logistics Optimization

Logistics has always generated data, but until recently most of that data was stored, rarely analysed, and almost never used to drive decisions in real time. The contemporary shift towards data-driven operations means that every order, every kilometre travelled, and every minute spent waiting becomes a data point that can be examined for patterns. Descriptive analytics answers the question "what happened?" by summarizing past events using statistics such as mean, median, minimum, maximum, and proportions. Diagnostic analytics answers the question "why did it happen?" by relating outcomes to explanatory variables such as distance, traffic, and weather. Predictive analytics answers "what is likely to happen?" by building models that estimate future outcomes. Prescriptive analytics answers "what should we do?" by recommending actions. This project is firmly positioned in the descriptive and diagnostic categories, while the Future Scope section outlines how it may be extended into predictive and prescriptive territory.

For a logistics company, analytics creates value in at least four ways. It enables better ETA estimation, which reduces customer disappointment. It enables smarter courier allocation, which reduces idle time and congestion. It informs pricing decisions, such as surge fees during peak periods or long-distance surcharges. Finally, it supports strategic planning, such as deciding where to place dark kitchens or how many couriers to recruit for a given time slot.

1.4 Python, Pandas, NumPy, and Seaborn in Logistics Analytics

Python has become the dominant language for data analytics because of its readable syntax, its large community, and its rich ecosystem of open-source libraries. Four libraries form the technical backbone of this project.

Pandas provides the DataFrame, a two-dimensional labelled data structure that resembles a spreadsheet or a database table. Using Pandas, a CSV file can be loaded in a single line, missing values can be detected with `isnull()`, columns can be filtered and transformed, and group-wise summaries can be generated using `groupby()`. In this project Pandas is responsible for data ingestion, cleaning, imputation, creation of the boolean feature *Is_Delayed*, and calculation of summary statistics.

NumPy supplies fast, vectorized numerical operations on arrays. Although Pandas is built on top of NumPy, direct use of NumPy is helpful for operations such as computing medians, handling NaN (Not a Number) values, and performing conditional logic with `np.where()`.

Matplotlib is the foundational plotting library in Python. It provides figure and axes objects, fine control over labels, titles, and reference lines, and the ability to save figures at publication quality.

Seaborn is a statistical visualization library built on Matplotlib. It simplifies the creation of histograms with kernel density estimates, scatter plots with categorical colouring, bar charts, and box plots. Seaborn's themes, such as `whitegrid`, give the charts a clean, professional appearance suitable for academic reports. The four visualizations presented in this report (histogram with KDE, distance scatter plot, time-of-day bar chart, and traffic box plot) were all produced using Seaborn and Matplotlib.

Together, these tools allow a student team to carry out an analysis that would otherwise require expensive commercial software, and they provide a bridge towards advanced machine-learning libraries such as scikit-learn, which are discussed in the Future Scope.

1.5 Scope and Organization of the Report

This report documents the complete life cycle of the project titled *Food Delivery Time and Order Analysis* (Project ID: DA-005), carried out by Group BCA_AI_05 of the BCA AI programme at Invertis University, Bareilly, under the guidance of Mr. Vishnu. The analysis is based on a Kaggle dataset of 1,000 food delivery orders described by nine columns. Section 2 states the problem; Section 3 lists the objectives; Section 4 describes the dataset and preprocessing methodology; Section 5 reviews the literature; Section 6 explains the working of the project; Section 7 presents the architecture; Section 8 provides the full source code; Section 9 presents results and a detailed discussion of four visual analyses; Section 10 outlines future scope; Section 11 concludes the report; and the final section lists the references.

2. Problem Statement:

2.1 Overview of the Problem

Food delivery platforms promise speed, yet the data analysed in this project shows that a large majority of orders take longer than the 45-minute benchmark. The problem addressed by this project can be stated formally as follows:

> *Given a historical dataset of food delivery orders described by distance, weather, traffic level, time of day, vehicle type, preparation time, courier experience, and delivery time, identify the principal operational factors that contribute to delivery delays, quantify the extent of SLA breach, and present the findings through clear statistical summaries and visualizations that can guide corrective action.*

The central difficulty is that delivery time is not controlled by a single factor. It emerges from the interaction of physical distance, human behaviour, infrastructure, environment, and platform decisions. Without systematic analysis, managers tend to rely on intuition, anecdotal complaints, or aggregate averages that conceal important variation. For example, an average delivery time of 56.73 minutes says nothing about whether the delay is evenly spread across the day or concentrated in a few congested hours, nor whether it is caused primarily by long distances or by heavy traffic.

2.2 Operational Bottlenecks

2.2.1 Distance Escalation Beyond 10 Kilometres

Distance is the most intuitive determinant of delivery time. As the distance between restaurant and customer grows, the courier must spend more time on the road, cross more intersections, and encounter more unpredictable events. However, the relationship is not merely linear. Beyond approximately 10 kilometres, the proportion of delayed orders rises sharply, and the scatter of delivery times widens considerably. A short-distance order of under 3 kilometres is generally predictable, while a long-distance order of 15 to 20 kilometres can take anywhere from around 50 to well over 100 minutes depending on conditions. Long distances also reduce the courier's ability to serve additional orders, which reduces overall fleet productivity and may increase waiting times for other customers.

2.2.2 Peak-Hour Congestion in the Morning and Evening

Demand for food is highly time-dependent. Lunchtime and dinnertime see surges in orders, and these surges coincide with the commuting peaks when roads are most crowded. The result is a double burden: more orders for the kitchen to prepare and more vehicles competing for road space. In the dataset, the Morning, Afternoon, and Evening periods each carry roughly three hundred orders, whereas the Night period carries far fewer. The delay share in the three busy periods (roughly 66 to 68 percent) is modestly higher than at night (roughly 61 percent), and because those periods carry far more orders, they account for most of the delayed orders, indicating that congestion and demand pressure are meaningful bottlenecks.

2.2.3 Weather and Traffic Effects

Adverse weather such as rain, fog, or snow reduces visibility and road grip, forcing couriers to ride more slowly and carefully. Weather also affects the availability of couriers, because some choose not to work in poor conditions. Traffic level, categorized in the dataset as Low, Medium, and High, captures the density of vehicles on the route. High traffic increases the median delivery time and raises the minimum time needed to complete an order, as visible in the box plot analysis presented in Section 9. When weather and traffic worsen together, delays compound.

2.3 Cost of Delayed Orders

Delayed orders impose a measurable financial burden. Consider a platform that processes thousands of orders per day. If two-thirds of orders breach the SLA and even a small fraction of those customers receive compensation, the cumulative cost becomes substantial. Costs include compensation vouchers, refunds for spoiled food, higher customer-support workload, and lost future revenue from customers who churn. In addition, restaurants may reduce their participation on the platform if they perceive that late deliveries damage their reputation.

2.4 Driver Burnout and Workforce Strain

The delivery partner is the human face of the platform, and the pressure to meet unrealistic time targets can lead to burnout. Long distances in heavy traffic increase physical fatigue, mental stress, and exposure to road accidents. High turnover among couriers imposes recruitment and training costs and reduces the proportion of experienced couriers, which in turn increases average delivery times. Understanding which conditions are systematically associated with extreme delays allows platforms to set fairer targets and to adjust incentives.

2.5 SLA Breach Penalties

Where platforms have contractual commitments to restaurants, corporate clients, or premium subscribers, SLA breaches may carry explicit penalties. Even where no formal penalty exists, repeated breaches reduce the platform's standing in the eyes of regulators, consumers, and investors. Quantifying the SLA breach rate, as done in this project, is therefore a prerequisite for any serious attempt at service improvement.

2.6 Research Gap Addressed by This Project

While large platforms possess proprietary analytics, student and small-business projects often lack structured, reproducible methods for analysing delivery delays. This project fills that gap by demonstrating a transparent pipeline, from raw CSV to actionable insight, using only open-source tools and a publicly available dataset.

3. Objectives of the Project:

3.1 Primary Objective

The primary objective of the project is to analyse a dataset of 1,000 food delivery orders in order to identify the factors that most strongly influence delivery duration and to measure the extent to which orders breach the 45-minute SLA.

3.2 Quantitative Objectives

1. To compute the central tendency and spread of delivery time, including mean, median, minimum, and maximum values.
2. To determine the proportion of orders classified as delayed using the rule *Delivery_Time_min > 45*.
3. To quantify differences in delivery time across Low, Medium, and High traffic levels.
4. To quantify the number of total and delayed orders in each time-of-day category (Morning, Afternoon, Evening, Night).
5. To examine the relationship between distance in kilometres and delivery time in minutes.

3.3 Imputation Targets

The raw dataset contains 30 missing entries in each of four columns. The imputation objectives are:

Column	Data Type	Imputation Technique	Reason
Weather	Categorical	Mode imputation	Most frequent category preserves distribution
Traffic_Level	Categorical	Mode imputation	Mean is undefined for categories
Time_of_Day	Categorical	Mode imputation	Maintains valid category labels
Courier_Experience_yrs	Numerical	Median imputation	Robust to skew and outliers

The target after imputation is a dataset with **zero missing values** in all nine columns, enabling consistent analysis without discarding any of the 1,000 records.

3.4 KPI Benchmarks

The Key Performance Indicators (KPIs) used to evaluate delivery performance are listed below.

KPI	Definition	Value Observed
Mean Delivery Time	Arithmetic average of Delivery_Time_min	56.73 minutes
Median Delivery Time	Middle value of sorted delivery times	55.50 minutes
Minimum Delivery Time	Fastest recorded order	8 minutes
Maximum Delivery Time	Slowest recorded order	153 minutes
Delay Proportion	Share of orders with time above 45 minutes	66.4 percent
SLA Threshold	Benchmark for classification	45 minutes

3.5 Visual Analytics Scope

The visual analytics objective is to produce four complementary charts arranged in a single dashboard figure: (1) a histogram with kernel density estimate showing the distribution of delivery time and the SLA line; (2) a scatter plot of distance against delivery time with delayed orders highlighted; (3) a stacked bar chart of total and delayed orders by time of day; and (4) a box plot comparing delivery time across traffic levels. Each chart is designed to answer one specific diagnostic question.

3.6 Learning and Skill Objectives

Beyond the analytical goals, the project aims to develop practical competence in the data-science workflow. Team members are expected to understand data ingestion, cleaning, imputation logic, feature engineering, statistical summarization, visualization design, version control through GitHub, technical documentation, and oral presentation of findings.

4. Methodology and Data Preprocessing:

4.1 Dataset Specification

The dataset used in this project is the publicly available *Food Delivery Times* dataset from Kaggle, stored in the file `Food_Delivery_Times.csv`. It contains **1,000 rows**, each representing one delivery order, and **nine columns** describing the order, its context, and its outcome.

No.	Column Name	Type	Description
1	Order_ID	Identifier	Unique number for each order
2	Distance_km	Numerical	Distance between restaurant and customer in kilometres
3	Weather	Categorical	Clear, Rainy, Foggy, Snowy, or Windy
4	Traffic_Level	Categorical	Low, Medium, or High
5	Time_of_Day	Categorical	Morning, Afternoon, Evening, or Night
6	Vehicle_Type	Categorical	Bike, Scooter, or Car
7	Preparation_Time_min	Numerical	Time taken by the restaurant to prepare the food
8	Courier_Experience_yrs	Numerical	Years of experience of the delivery partner
9	Delivery_Time_min	Numerical	Total delivery time in minutes (the target variable of analysis)

The distances in the data range from under one kilometre to about twenty kilometres, as visible on the horizontal axis of the scatter plot, and delivery times range from 8 to 153 minutes.

4.2 Overall Methodological Framework

The methodology follows the standard data-analysis pipeline: data acquisition, inspection, cleaning, feature engineering, descriptive statistics, visualization, and interpretation. Each stage is implemented as a clearly separated block of Python code in `main.py`, so that the pipeline is reproducible and every transformation can be audited.

4.3 Data Inspection

After loading the CSV file with `pd.read_csv()`, the first step is inspection. The methods `head()`, `info()`, `describe()`, and `isnull().sum()` reveal the structure, data types, summary statistics, and the number of missing values per column. Inspection is essential because the choice of imputation strategy depends on whether a column is numerical or categorical and on how its values are distributed.

4.4 Missing Value Handling

Missing data is one of the most common problems in real-world datasets. Inspection of the dataset shows that exactly four columns, namely Weather, Traffic_Level, Time_of_Day, and Courier_Experience_yrs, contain missing entries, 30 in each column (3 percent of the 1,000 records per column). Deleting every row with a missing value would shrink the dataset and could introduce bias, particularly because missing entries in different columns occur in different rows. Imputation, meaning the replacement of missing values with plausible estimates, preserves all 1,000 records.

4.4.1 Mode Imputation for Categorical Fields

For categorical variables, the arithmetic mean has no meaning, because one cannot average the words "Rainy" and "Clear". The most suitable simple estimate is the **mode**, the category that occurs most frequently.

Mathematical definition. Let a categorical column *X* take values from a finite set of categories {c1, c2, ..., ck}. Let *nj* denote the number of observed (non-missing) entries equal to category cj. The mode is

> *Mode(X) = cm, where m = argmax over j of nj*

Every missing entry of *X* is then replaced by Mode(X).

Step-by-step practical procedure.

1. Select the column, for example Weather.
2. Ignore missing values and count the frequency of each category using `value_counts()`.
3. Identify the category with the highest count; this is the mode.
4. Use `fillna()` to replace each missing entry with that mode.
5. Verify that `isnull().sum()` now returns zero for the column.

In Pandas, `df['Weather'].mode()[0]` returns the first mode, and `df['Weather'].fillna(mode_value)` performs the replacement. For this dataset the modes are **Clear** for Weather (470 observed orders), **Medium** for Traffic_Level (390 observed orders), and **Morning** for Time_of_Day (308 observed orders). The same process is repeated for each of the three columns.

Advantages. Mode imputation is simple, fast, and guarantees that the imputed value is a valid category. **Limitations.** It slightly inflates the frequency of the most common category and ignores relationships between variables. Because only 3 percent of the entries are missing in each column, the distortion is minimal.

4.4.2 Median Imputation for Numerical Fields

For numerical columns, either the mean or the median could be used. The **median** is preferred when the distribution is skewed or contains outliers, because it is resistant to extreme values.

Mathematical definition. Let the non-missing observations of a numerical column be sorted as x(1) <= x(2) <= ... <= x(n). The median is

> *Median = x((n+1)/2) if n is odd, and Median = (x(n/2) + x(n/2+1)) / 2 if n is even.*

Every missing value in Courier_Experience_yrs is replaced with this median, which equals **5.0 years** for this dataset.

Step-by-step practical procedure.

1. Compute `median_value = df['Courier_Experience_yrs'].median()`; Pandas automatically ignores NaN values.
2. Apply `df['Courier_Experience_yrs'].fillna(median_value)`.
3. Confirm that no missing values remain.

Justification. Courier experience is measured in years and is naturally right-skewed, since a few couriers have many years of experience while most have fewer. The mean would be pulled upwards by the experienced minority, whereas the median represents a typical courier more faithfully.

4.5 Boolean Feature Engineering

Feature engineering is the creation of new variables from existing ones in order to make patterns easier to detect. The key engineered feature in this project is the boolean column **Is_Delayed**, defined as

> *Is_Delayed = (Delivery_Time_min > 45)*

The value is `True` when the order exceeded the SLA threshold and `False` otherwise. In Pandas this is a one-line vectorized operation: `df['Is_Delayed'] = df['Delivery_Time_min'] > 45`. Because Python treats `True` as 1 and `False` as 0, the mean of this column is directly the proportion of delayed orders, which gives the headline KPI of 66.4 percent. The column is also used as the colour variable in the scatter plot and as the splitting variable when computing delayed counts per time-of-day category.

4.6 Descriptive Statistics

Descriptive statistics condense the delivery-time column into a few interpretable numbers: mean (56.73 minutes), median (55.50 minutes), minimum (8 minutes), and maximum (153 minutes). The closeness of mean and median indicates a distribution that is only mildly skewed, although the long right tail visible in the histogram, extending past 140 minutes, shows that rare but extreme delays exist.

4.7 Visualization Design

The visualization stage uses a 2 by 2 grid of subplots in one figure, saved as a high-resolution PNG. The Seaborn style `whitegrid` is used for readability. A red dashed reference line at 45 minutes is drawn on the histogram, scatter plot, and box plot so that the SLA benchmark is visible across charts. Colour choices are deliberate: teal for neutral distributions, green for on-time and red for delayed orders, and a three-colour palette for the three traffic levels.

4.8 Reproducibility and Version Control

The complete code and dataset reference are stored in a GitHub repository set up by the technical head. Version control provides a history of changes, supports collaboration, and allows the group and the mentor to reproduce the results exactly.

5. Literature Review and References:

5.1 Introduction to the Literature

The study of delivery time and logistics performance sits at the intersection of operations research, statistics, transportation engineering, and machine learning. Over several decades, researchers have developed formal models for routing vehicles, estimating travel durations, forecasting demand, and profiling delays. This section reviews representative work in four areas that are directly relevant to the present project: supply chain logistics and routing, dynamic ETA prediction, statistical delay profiling, and data-handling and visualization methodology. Formal IEEE-style references are given in Section 12 at the end of this report.

5.2 Supply Chain Logistics and Vehicle Routing

The mathematical study of vehicle routing began with the seminal work of Dantzig and Ramser, who formulated the "truck dispatching problem" as the task of assigning a fleet of vehicles to serve customers at minimum total travel cost [1]. This formulation, now known as the Vehicle Routing Problem (VRP), became the foundation of a vast literature. Subsequent variants introduced time windows, vehicle capacities, multiple depots, and stochastic demand. A comprehensive treatment of problems, exact methods, and heuristics is provided in the edited volume by Toth and Vigo [2]. The key lesson from this literature is that route length and the number of stops directly influence service time, and that optimal routing is computationally difficult, which has motivated heuristic and metaheuristic approaches in practice.

Shortest-path computation is an essential building block of routing. Dijkstra's classic algorithm for finding shortest paths in weighted graphs [3] remains the conceptual basis of many modern navigation engines, although practical systems use accelerated variants and live traffic data. For food delivery in particular, the Meal Delivery Routing Problem studied by Reyes and co-authors highlights the unique challenges of the domain: orders arrive dynamically, food is perishable, and couriers must be matched to orders under tight time limits [4]. Unlike classical parcel logistics, meal delivery requires decisions in minutes, which explains why distance and congestion exert such a strong influence on delay.

5.3 Dynamic ETA Prediction

Estimated time of arrival is the customer-facing outcome of the entire logistics process. Research on travel-time estimation has progressed from simple averages, through regression models, to deep-learning architectures. Wang, Fu, and Ye proposed a method that learns to estimate travel time directly from origin-destination information and historical trips, showing that learned models can outperform route-based estimation [5]. Li and co-authors extended this line of work by using multi-task representation learning to improve travel-time estimation accuracy [6]. These studies illustrate that features such as distance, time of day, and traffic conditions, the same variables present in our dataset, are central to accurate ETA prediction.

On the machine-learning side, ensemble methods have proved particularly effective for tabular data. Breiman's random forests combine many decision trees to reduce variance and capture non-linear interactions [7], while gradient-boosted trees as implemented in XGBoost provide scalable and highly accurate prediction on structured datasets [8]. The open-source scikit-learn library has made these techniques accessible to students and practitioners alike [9]. The present project does not build predictive models, but the descriptive analysis performed here is the natural first step, since it reveals which variables deserve to be included as predictors.

5.4 Statistical Delay Profiling

Delay profiling is concerned with describing how and why delays occur. The foundation of this practice is exploratory data analysis, popularized by Tukey, who advocated visual and numerical summaries such as box plots and stem-and-leaf displays as tools for discovering structure in data before formal modelling [10]. The box plot used in Section 9 of this report is a direct descendant of Tukey's work. Kernel density estimation, which produces the smooth curve overlaid on the histogram, is described in detail by Silverman [11], who explains how the choice of bandwidth controls the smoothness of the estimate. For broader statistical learning concepts, including regression, classification, and resampling, the textbook of Hastie, Tibshirani, and Friedman remains a standard reference [12].

A recurring issue in delay profiling is the presence of missing data. Little and Rubin provide the standard theoretical treatment of missing-data mechanisms, namely missing completely at random, missing at random, and missing not at random, and discuss the strengths and weaknesses of simple imputation such as mean, median, or mode replacement versus more sophisticated multiple imputation [13]. Their analysis explains why simple imputation is acceptable when the proportion of missing values is small, and why the median is preferred for skewed numerical variables.

5.5 Tools and Visualization Methodology

The software tools used in this project are well documented in the scientific literature. Pandas, introduced by McKinney, provides the DataFrame structure for statistical computing in Python [14]. NumPy supplies array programming capabilities that underpin the entire scientific Python ecosystem [15]. Matplotlib, described by Hunter, is the foundational two-dimensional plotting library [16], and Seaborn, described by Waskom, offers a high-level interface for statistical graphics [17]. These tools are open-source, widely cited, and suitable for reproducible research.

5.6 Synthesis and Research Gap

The reviewed literature shows that distance, time of day, and traffic are repeatedly identified as determinants of travel and delivery time, that exploratory and visual techniques are the recommended starting point for understanding such determinants, and that simple imputation is a defensible choice for small amounts of missing data. However, most published work either addresses large proprietary datasets or focuses on complex optimization and deep-learning models that are beyond the scope of an undergraduate project. The present work contributes a transparent, reproducible, entry-level analysis of a publicly available dataset, with a clear link between descriptive findings and managerial recommendations.

6. Explanation of Project (Working):

This section explains, step by step, how the project operates from the moment the raw CSV file is read until the final insights are produced. The pipeline has seven stages.

6.1 Stage 1: Data Ingestion from CSV

The workflow begins with the file `Food_Delivery_Times.csv`. The function `pd.read_csv()` reads the file and converts each row into a record of a Pandas DataFrame. At this point the DataFrame contains 1,000 rows and nine columns. Missing entries appear as NaN (Not a Number) markers. The program immediately prints the first few rows and the structural information, which allows the team to verify that the file was loaded correctly, that column names match expectations, and that numerical columns were not accidentally read as text.

6.2 Stage 2: Data Quality Assessment and Preprocessing

The second stage counts missing values per column using `df.isnull().sum()`. The output identifies the four columns requiring imputation, each with 30 missing entries. The program then applies mode imputation to the three categorical columns (Weather, Traffic_Level, Time_of_Day) and median imputation to the numerical column Courier_Experience_yrs, exactly as described in Section 4. After imputation, the program re-checks the missing counts to confirm that every column is complete. This verification step is an example of defensive programming: rather than assuming that the cleaning worked, the code proves it.

6.3 Stage 3: Feature Engineering and SLA Filtering

In the third stage the boolean column Is_Delayed is created by comparing Delivery_Time_min with the 45-minute threshold. This single column enables SLA filtering: `df[df['Is_Delayed']]` returns only the delayed orders, while `df[~df['Is_Delayed']]` returns the on-time orders. The count of delayed orders divided by the total number of orders gives the delay proportion of 66.4 percent. The threshold is stored in a named constant (`SLA_THRESHOLD = 45`) so that it can be changed in one place if management wishes to test a different benchmark such as 30 or 60 minutes.

6.4 Stage 4: Descriptive Statistics

The fourth stage computes the mean, median, minimum, and maximum of delivery time, together with the delay proportion. Pandas provides these through `.mean()`, `.median()`, `.min()`, and `.max()`. The results are printed in a formatted KPI summary: mean 56.73 minutes, median 55.50 minutes, minimum 8 minutes, maximum 153 minutes, and delay proportion 66.4 percent. A grouped summary is also created, in which orders are grouped by Time_of_Day and the total and delayed counts are computed. This grouped table is the data source for the bar chart.

6.5 Stage 5: Visual Dashboard Compilation

In the fifth stage the program creates a figure containing four subplots. The histogram with KDE displays the distribution of delivery time. The scatter plot displays distance against delivery time with colour-coded delay status. The stacked bar chart displays total and delayed orders for each time-of-day category. The box plot displays delivery-time distributions for Low, Medium, and High traffic. Titles, axis labels, legends, and the red dashed 45-minute line are added. The completed figure is saved as `real_food_delivery_visualizations.png` at high resolution so that it can be inserted into reports and presentations.

6.6 Stage 6: Interpretation and Insight Generation

The sixth stage is human interpretation. The team examines each chart and extracts conclusions: delivery time is centred near 55 minutes and not near 45; distance shows a clear positive relationship with delivery time; busy periods have larger order volumes and high delay counts; and high traffic shifts the entire distribution upwards. These insights are recorded in the Result and Discussion section.

6.7 Stage 7: Actionable Recommendations

Finally, the insights are translated into recommendations such as revising ETAs for long distances, adding courier capacity at peak hours, using traffic-aware dispatching, and introducing distance-based pricing or geofenced delivery caps. These recommendations connect the analysis to practical decision-making and form the bridge to the Future Scope.

6.8 Summary of the Pipeline

Stage	Input	Process	Output
1	CSV file	Load with Pandas	Raw DataFrame (1,000 x 9)
2	Raw DataFrame	Mode and median imputation	Complete DataFrame
3	Complete DataFrame	Create Is_Delayed using 45-minute rule	Labelled DataFrame
4	Labelled DataFrame	Descriptive statistics and grouping	KPI table and summary
5	KPI data and DataFrame	Seaborn and Matplotlib plotting	Four-panel dashboard PNG
6	Dashboard	Human interpretation	Insights
7	Insights	Strategic reasoning	Recommendations

7. System Architecture and Workflow Diagram:

7.1 Architectural Overview

The project follows a layered architecture in which each layer has a single responsibility and communicates only with the adjacent layers. This separation of concerns makes the system easier to understand, test, and extend. Five layers are defined: the Data Layer, the Processing Layer, the Analytics Layer, the Application Layer, and the Action Layer.

7.2 Layered Architecture Diagram

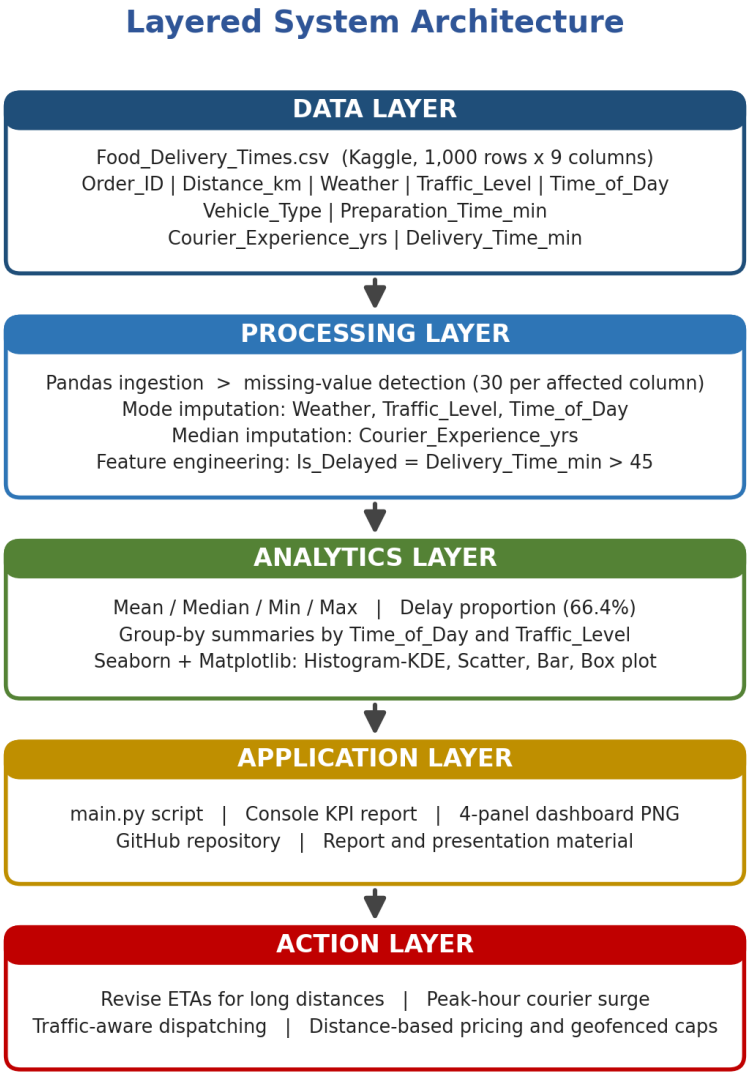


Figure 2: Layered system architecture of the project

7.3 Description of Each Layer

Data Layer. This is the foundation and contains the raw CSV file. It represents the output of the real-world delivery process. No transformation is applied here; the raw file is treated as read-only, so the original evidence is never altered.

Processing Layer. This layer is responsible for cleaning and preparing the data. It performs missing-value detection, mode and median imputation, and the creation of the Is_Delayed feature. The output is a complete and analysis-ready DataFrame.

Analytics Layer. This layer computes statistics and builds visualizations. It transforms the clean data into numbers and charts that can be interpreted by humans.

Application Layer. This layer packages the analysis into usable artefacts: the executable `main.py` script, the printed KPI report, the dashboard image, and the GitHub repository that stores everything.

Action Layer. The top layer converts insight into operational decisions. It does not belong to the code, but represents the managerial use of the results.

7.4 Workflow Diagram



Figure 3: End-to-end workflow of the analysis pipeline

7.5 Design Principles

Three design principles guided the architecture. First, **reproducibility**: running the script on the same CSV always yields the same outputs. Second, **transparency**: every transformation is a visible line of code with comments. Third, **extensibility**: new layers, such as a machine-learning model, can be added without disturbing the existing ones.

8. Program Source Code:

8.1 Overview of the Script

The complete implementation is contained in a single Python file, `main.py`, developed by the Technical Head, Sudheer Kumar. The script is organized into numbered blocks with extensive inline comments. To run it, Python 3.9 or later is required, along with the libraries pandas, numpy, matplotlib, and seaborn, which can be installed using `pip install pandas numpy matplotlib seaborn`. The file `Food_Delivery_Times.csv` must be placed in the same folder as the script.

8.2 Full Source Code (main.py)

```
# ======================================================================
# Project  : Food Delivery Time and Order Analysis (Project ID: DA-005)
# Group    : BCA_AI_05 | Invertis University, Bareilly (CSED)
# Author   : Sudheer Kumar (Technical Head)
# Mentor   : Mr. Vishnu
# Purpose  : Clean the delivery dataset, compute KPIs, and build a
#            4-panel visual dashboard of delivery delays.
# ======================================================================

# ----------------------------------------------------------------------
# BLOCK 1: IMPORT LIBRARIES
# ----------------------------------------------------------------------
import pandas as pd              # DataFrames: loading, cleaning, grouping
import numpy as np               # Numerical helpers (NaN handling, math)
import matplotlib.pyplot as plt  # Base plotting: figures, axes, saving
import seaborn as sns            # Statistical plots built on Matplotlib

# ----------------------------------------------------------------------
# BLOCK 2: GLOBAL SETTINGS
# ----------------------------------------------------------------------
SLA_THRESHOLD = 45               # Orders slower than this (minutes) are delayed
CSV_FILE = "Food_Delivery_Times.csv"
OUTPUT_IMAGE = "real_food_delivery_visualizations.png"

sns.set_theme(style="whitegrid") # Clean grid background for all charts

# ----------------------------------------------------------------------
# BLOCK 3: LOAD AND INSPECT DATA
# ----------------------------------------------------------------------
df = pd.read_csv(CSV_FILE)       # Read the CSV into a DataFrame

print("Shape (rows, columns):", df.shape)   # Expect (1000, 9)
print("\nFirst five rows:")
print(df.head())                            # Quick visual check
print("\nColumn data types:")
print(df.dtypes)                            # Confirm numeric vs text
print("\nMissing values BEFORE imputation:")
print(df.isnull().sum())                    # NaN count per column

# ----------------------------------------------------------------------
# BLOCK 4: MISSING VALUE IMPUTATION
# ----------------------------------------------------------------------
# 4a. Categorical columns -> MODE (most frequent category)
categorical_cols = ["Weather", "Traffic_Level", "Time_of_Day"]
for col in categorical_cols:
    mode_value = df[col].mode()[0]          # Most frequent label
    df[col] = df[col].fillna(mode_value)    # Replace NaN with the mode
    print(f"Imputed {col} with mode: {mode_value}")

# 4b. Numerical column -> MEDIAN (robust to skew and outliers)
median_exp = df["Courier_Experience_yrs"].median()   # NaN ignored
df["Courier_Experience_yrs"] = df["Courier_Experience_yrs"].fillna(median_exp)
print(f"Imputed Courier_Experience_yrs with median: {median_exp}")

# 4c. Verification: every column should now report zero missing values
print("\nMissing values AFTER imputation:")
print(df.isnull().sum())

# ----------------------------------------------------------------------
# BLOCK 5: FEATURE ENGINEERING
# ----------------------------------------------------------------------
```

```
# Boolean flag: True when delivery exceeded the SLA threshold
df["Is_Delayed"] = df["Delivery_Time_min"] > SLA_THRESHOLD

# ---------------------------------------------------------------------
# BLOCK 6: DESCRIPTIVE STATISTICS (KPIs)
# ---------------------------------------------------------------------
mean_time = df["Delivery_Time_min"].mean()      # Arithmetic average
median_time = df["Delivery_Time_min"].median()  # Middle value
min_time = df["Delivery_Time_min"].min()        # Fastest order
max_time = df["Delivery_Time_min"].max()        # Slowest order
delay_pct = df["Is_Delayed"].mean() * 100       # True=1, False=0 -> share

print("\n========== KEY PERFORMANCE INDICATORS ==========")
print(f"Mean delivery time   : {mean_time:.2f} min")
print(f"Median delivery time : {median_time:.2f} min")
print(f"Minimum              : {min_time} min")
print(f"Maximum              : {max_time} min")
print(f"Delayed orders       : {delay_pct:.1f}% (> {SLA_THRESHOLD} min)")

# Median delivery time for each traffic level (supports Chart 4)
print("\nMedian delivery time by traffic level:")
print(df.groupby("Traffic_Level")["Delivery_Time_min"].median())

# ---------------------------------------------------------------------
# BLOCK 7: GROUPED SUMMARY BY TIME OF DAY (data for Chart 3)
# ---------------------------------------------------------------------
time_order = ["Morning", "Afternoon", "Evening", "Night"]

time_summary = (
    df.groupby("Time_of_Day")
      .agg(Total_Orders=("Order_ID", "count"),    # all orders in the slot
           Delayed_Orders=("Is_Delayed", "sum"))  # delayed orders only
      .reindex(time_order)                        # fixed logical order
      .reset_index()
)
print("\nOrders by time of day:")
print(time_summary)

# ---------------------------------------------------------------------
# BLOCK 8: VISUAL DASHBOARD (2 x 2 GRID)
# ---------------------------------------------------------------------
fig, axes = plt.subplots(2, 2, figsize=(14, 10))   # 4 subplots in 1 figure

# ---- Chart 1: Histogram + KDE of delivery time ----------------------
sns.histplot(df["Delivery_Time_min"], bins=30, kde=True,
             color="teal", ax=axes[0, 0])
axes[0, 0].axvline(SLA_THRESHOLD, color="red", linestyle="--",
                   label="Delay Threshold (45 min)")   # SLA reference line
axes[0, 0].set_title("1. Delivery Time Distribution", fontweight="bold")
axes[0, 0].set_xlabel("Delivery Time (minutes)")
axes[0, 0].set_ylabel("Order Count")
axes[0, 0].legend()

# ---- Chart 2: Distance vs Delivery Time scatter ---------------------
sns.scatterplot(data=df, x="Distance_km", y="Delivery_Time_min",
                hue="Is_Delayed",                       # colour by status
                palette={False: "green", True: "red"},  # green=on time
                alpha=0.7, ax=axes[0, 1])
axes[0, 1].axhline(SLA_THRESHOLD, color="red", linestyle="--")
axes[0, 1].set_title("2. Distance vs Delivery Time", fontweight="bold")
axes[0, 1].set_xlabel("Distance (km)")
axes[0, 1].set_ylabel("Delivery Time (minutes)")

# ---- Chart 3: Orders by time of day (total vs delayed) --------------
# Draw the TOTAL bar first (light), then overlay DELAYED (dark) so the
# visible light segment represents on-time orders.
sns.barplot(data=time_summary, x="Time_of_Day", y="Total_Orders",
            color="lightblue", label="On-Time Orders", ax=axes[1, 0])
sns.barplot(data=time_summary, x="Time_of_Day", y="Delayed_Orders",
            color="crimson", label="Delayed Orders", ax=axes[1, 0])
axes[1, 0].set_title("3. Time of Day Volume & Delays", fontweight="bold")
axes[1, 0].legend()

# ---- Chart 4: Box plot by traffic level -----------------------------
sns.boxplot(data=df, x="Traffic_Level", y="Delivery_Time_min",
            order=["Low", "Medium", "High"],            # logical order
            hue="Traffic_Level", palette="Set2",
            legend=False, ax=axes[1, 1])
axes[1, 1].axhline(SLA_THRESHOLD, color="red", linestyle="--")
axes[1, 1].set_title("4. Traffic Level Impact on Delivery Time",
                     fontweight="bold")
```

```
# ----------------------------------------------------------------------
# BLOCK 9: SAVE AND DISPLAY
# ----------------------------------------------------------------------
plt.tight_layout()                                  # avoid overlaps
plt.savefig(OUTPUT_IMAGE, dpi=300, bbox_inches="tight")  # high resolution
plt.show()                                          # open the window
print(f"\nDashboard saved as {OUTPUT_IMAGE}")
```

8.3 Explanation of the Code Blocks

Block 1 (Imports). Four libraries are imported using conventional aliases. Pandas handles tabular data, NumPy supports numerical work, Matplotlib creates figures, and Seaborn provides statistical chart types.

Block 2 (Global settings). Constants are defined once at the top so that the threshold, input file, and output file can be modified without searching through the code. The Seaborn theme `whitegrid` is activated.

Block 3 (Load and inspect). The CSV is read and basic checks are printed. The expected shape is 1,000 rows by 9 columns.

Block 4 (Imputation). A loop applies mode imputation to each categorical column. Median imputation is applied to the numerical column. A final verification prints the missing-value counts, which should all be zero.

Block 5 (Feature engineering). The boolean Is_Delayed column is created through a vectorized comparison.

Block 6 (KPIs). Mean, median, minimum, maximum, and delay percentage are computed. Because boolean values behave as 0 and 1, the mean of Is_Delayed multiplied by 100 is the delay percentage.

Block 7 (Grouped summary). The data is grouped by Time_of_Day. The `agg()` function computes counts and sums, and `reindex()` imposes the logical order Morning, Afternoon, Evening, Night instead of alphabetical order.

Block 8 (Dashboard). A 2 by 2 grid of axes is created. Each chart is drawn on its own axes object, and the SLA reference line is added where relevant.

Block 9 (Save and display). `tight_layout()` prevents overlapping labels, `savefig()` writes a 300-dpi image, and `show()` displays it on screen.

8.4 Sample Program Output

When the script is run on the dataset, the following key results are printed to the console, confirming the figures used throughout this report:

```
Imputed Weather with mode: Clear
Imputed Traffic_Level with mode: Medium
Imputed Time_of_Day with mode: Morning
Imputed Courier_Experience_yrs with median: 5.0

========== KEY PERFORMANCE INDICATORS ==========
Mean delivery time   : 56.73 min
Median delivery time : 55.50 min
Minimum              : 8 min
Maximum              : 153 min
Delayed orders       : 66.4% (> 45 min)

Median delivery time by traffic level:
High      65.0
Low       51.0
Medium    54.0
```

8.5 Notes on Execution

If Seaborn reports an error regarding the `legend=False` argument in `boxplot`, the installed version is older than 0.13 and should be upgraded using `pip install --upgrade seaborn`.

9. Result and Discussions:

9.1 Overview of Results

This section presents the quantitative results of the analysis and then discusses each of the four visual graphs in detail. The complete four-panel dashboard produced by the project code is shown in Figure 1. Values quoted for the individual charts are read from this image, so counts read from the bars and box plots are approximate and are marked with the word "approximately".

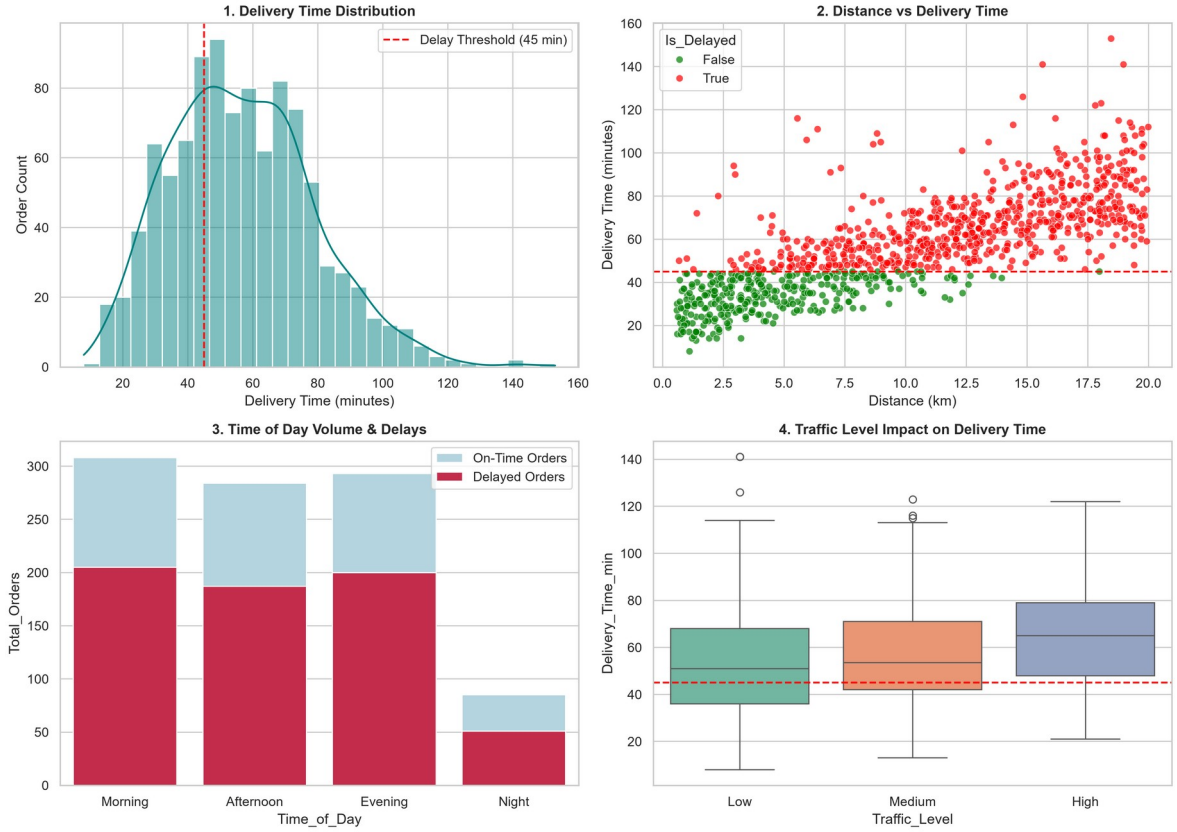


Figure 1: Four-panel visual dashboard (Delivery Time Analysis)

9.2 Key Performance Indicator Breakdown

KPI	Value	Interpretation
Mean delivery time	56.73 minutes	Average order takes almost 12 minutes more than the SLA
Median delivery time	55.50 minutes	Half of all orders take 55.5 minutes or longer
Minimum delivery time	8 minutes	Best-case, very short-distance order
Maximum delivery time	153 minutes	Extreme outlier, more than three times the SLA
Delay proportion	66.4 percent	About two out of every three orders breach the 45-minute SLA
SLA threshold	45 minutes	Benchmark used for the Is_Delayed flag

Discussion of the KPIs. The mean (56.73) and median (55.50) are separated by only about 1.2 minutes. When the mean is slightly larger than the median, the distribution has a mild right skew, meaning a

minority of very slow deliveries pulls the average upwards. The spread between the minimum of 8 minutes and the maximum of 153 minutes is a range of 145 minutes, which signals considerable variability in the service experience. Most importantly, the typical (median) order is already more than 10 minutes beyond the SLA, which means that late delivery is the normal condition in this dataset and not an exception. With 66.4 percent of orders delayed, only about one-third of customers receive their food within the benchmark.

Two interpretations are possible. Either the 45-minute SLA is too strict for the operating environment represented in the data, in which case the platform should publish more realistic ETAs, or the operation is genuinely under-performing, in which case process improvements are required. The visual analyses below help to separate these explanations by showing where the delays are concentrated.

9.3 Graph 1: Delivery Time Distribution (Histogram with KDE)

Description of the graph. The first chart is a histogram of Delivery_Time_min with 30 bins, overlaid with a smooth kernel density estimate in teal. A red dashed vertical line marks the 45-minute delay threshold. The horizontal axis shows delivery time in minutes from roughly 10 to 150, and the vertical axis shows the number of orders in each bin.

Shape of the distribution. The histogram is unimodal with a broad peak. The tallest bars occur between roughly 45 and 55 minutes, with the single highest bar containing approximately 94 orders. The KDE curve rises steadily from the left, reaches a first high point near 48 minutes, shows a slight plateau or secondary shoulder between about 60 and 70 minutes, and then declines. This shoulder suggests that the data may be a mixture of two kinds of orders: relatively quick short-distance deliveries and slower long-distance deliveries. After 80 minutes, the frequency falls quickly, and a long thin tail extends beyond 120 minutes up to the maximum of 153 minutes. A small isolated bar near 140 minutes represents extreme outliers.

Relationship with the SLA line. The red dashed line at 45 minutes falls on the left side of the peak. In other words, the bulk of the distribution lies to the right of the SLA threshold. The area to the left of the line, which represents on-time orders, is much smaller than the area to the right. This visual impression agrees exactly with the numerical result that 66.4 percent of orders are delayed.

Operational meaning. The position of the peak shows that the platform's typical performance is around 50 to 55 minutes, not 45. If the platform wishes to meet a 45-minute SLA reliably, it would need to shift the whole distribution to the left by about 10 minutes, which is a substantial improvement. Alternatively, it could redefine its promise. The right tail is also important: the few orders above 100 minutes produce disproportionate customer dissatisfaction and are strong candidates for root-cause analysis. Because they are rare, they do not change the median much, but they inflate the mean and damage customer trust.

Limitations. A histogram shows only the overall pattern and does not explain causes. For explanation, the remaining three graphs relate delivery time to distance, time of day, and traffic.

9.4 Graph 2: Distance versus Delivery Time (Scatter Plot)

Description of the graph. The second chart is a scatter plot with distance in kilometres on the horizontal axis (from about 0.5 to 20 km) and delivery time in minutes on the vertical axis. Each point is one order. Points are coloured green if the order was on time and red if it was delayed, and a red dashed horizontal line marks the 45-minute SLA.

Overall trend. There is an unmistakable positive relationship: as distance increases, delivery time increases. Orders under about 3 kilometres cluster near the bottom-left of the plot with delivery times mostly between 15 and 45 minutes, and most of these points are green. As distance grows to 5 to 10

kilometres, the cloud moves upward, and an increasing number of points sit above the SLA line. Beyond about 10 kilometres, the cloud lies almost entirely above 45 minutes, and green points become rare. For distances between 15 and 20 kilometres, typical delivery times range from about 50 to 120 minutes, and virtually every order is red.

Widening spread. The vertical scatter around the trend grows with distance. At 2 kilometres, delivery times differ by perhaps 30 minutes among orders, whereas at 18 kilometres they differ by more than 60 minutes. Statisticians call this pattern heteroscedasticity, meaning that variability is not constant. Operationally, it means that long-distance deliveries are not only slower on average but also far less predictable, which makes accurate ETAs harder to produce.

Anomalies. A few red points appear at short distances with high delivery times. For example, several orders at about 5 to 9 kilometres have delivery times above 100 minutes, and a handful of orders at about 2 to 3 kilometres take 80 to 95 minutes. These anomalies cannot be explained by distance alone and probably result from other factors such as heavy traffic, bad weather, long preparation time, or an inexperienced courier. They illustrate why a multi-factor analysis is necessary.

Operational meaning. The 10-kilometre mark emerges as a practical tipping point. Within 10 kilometres the platform has a reasonable chance of meeting a 45-minute SLA, particularly under 5 kilometres. Beyond 10 kilometres, the SLA is essentially unachievable under existing conditions. This supports policy options such as a geofenced maximum delivery radius, a distance-based delivery surcharge, a longer promised ETA for far orders, or stationing couriers closer to high-demand restaurants.

9.5 Graph 3: Time of Day Volume and Delays (Stacked Bar Chart)

Description of the graph. The third chart shows, for each of the four time-of-day categories, the total number of orders (light blue bar) and the number of delayed orders (crimson bar) drawn in front. The visible light-blue portion above the crimson bar therefore represents on-time orders.

Approximate values read from the chart.

Time of Day	Total Orders (approx.)	Delayed Orders (approx.)	Approx. Delay Share
Morning	307	205	67 percent
Afternoon	283	187	66 percent
Evening	292	200	68 percent
Night	84	51	61 percent

Observations. First, order volume is concentrated in three periods. Morning, Afternoon, and Evening carry roughly 280 to 310 orders each, while Night carries only about 84. Morning and Evening account for the two largest delay counts, with approximately 205 and 200 delayed orders respectively. Second, the delay share, that is, the fraction of orders that are delayed, is fairly uniform across the three busy periods at around two-thirds, and is somewhat lower at night at about 61 percent. Third, because the number of delayed orders is proportional to volume, the busiest periods account for the vast majority of the total delay burden.

Interpretation. The data does not show a dramatic difference in the delay *rate* between daytime periods; instead, the high delay rate is a system-wide condition, with night-time being modestly better, probably because roads are clearer and demand is lower. The key insight is therefore about *absolute* delay counts: Morning, Afternoon, and Evening generate the bulk of late orders, so interventions such as extra couriers, staggered shifts, and incentive pay should be concentrated at those times. The lower night-time volume and slightly better performance suggest that congestion and demand pressure play a

role, but the fact that even night-time delay is above 60 percent indicates that structural factors, notably distance, matter as well.

Data note. The four category totals read from the chart add up to about 970 orders, because 30 records have no recorded Time_of_Day and were not counted in this grouping. After mode imputation, those 30 records are assigned to the most frequent category, Morning. Running the script in this report after imputation therefore gives the following exact counts for all 1,000 orders:

Time of Day	Total Orders	Delayed Orders
Morning	338	226
Afternoon	284	187
Evening	293	200
Night	85	51

The qualitative conclusions are unchanged, but the group should state clearly which version of the data was used when presenting exact numbers.

9.6 Graph 4: Traffic Level Impact on Delivery Time (Box Plot)

Description of the graph. The fourth chart contains three box plots, one for each traffic level: Low (green), Medium (orange), and High (blue-purple). Each box spans the interquartile range (the middle 50 percent of orders) and has a line at the median. Whiskers extend to the most extreme non-outlier values, and individual circles denote outliers. A red dashed line at 45 minutes marks the SLA.

Approximate values read from the chart.

Traffic Level	Median	Lower Quartile	Upper Quartile	Lower Whisker	Upper Whisker
Low	about 51	about 36	about 68	about 8	about 114
Medium	about 53 to 54	about 42	about 71	about 13	about 113
High	about 65	about 48	about 79	about 21	about 122

Observations. The median delivery time rises steadily with traffic level: 51 minutes under Low traffic, 54 under Medium, and 65 under High (exact medians computed by the program). The difference between Low and High medians is 14 minutes, which is substantial. The whole box shifts upward with traffic: under High traffic the lower quartile is about 48 minutes, which is above the SLA line, meaning that more than 75 percent of High-traffic orders exceed 45 minutes. Even the fastest High-traffic delivery takes about 21 minutes, whereas the fastest Low-traffic delivery takes about 8 minutes, showing that heavy traffic raises the floor of delivery time as well as the median.

An important nuance. The medians of Low and Medium traffic are close, and both are already above the SLA line. This means that even in Low traffic the typical order is late. Traffic therefore worsens an existing problem but is not the sole cause. The wide boxes and long whiskers under all three categories confirm that other variables, particularly distance, are producing large variation within each traffic class.

Outliers. Outlier circles appear above the upper whisker for Low traffic (at about 126 and 141 minutes) and for Medium traffic (several points near 115 to 123 minutes). High traffic shows no outlier circles, probably because its whisker already extends to about 122 minutes. The extreme Low-traffic outliers imply that factors other than traffic, such as bad weather, vehicle breakdown, or long kitchen preparation, can produce very long deliveries even on empty roads.

Operational meaning. Real-time traffic information should be built into ETA calculation and dispatching. During high-traffic windows, the platform could add buffer time to quoted ETAs, prefer couriers with two-wheelers that move through congestion more easily, or avoid batching multiple orders on the same trip.

9.7 Integrated Discussion of All Four Graphs

Taken together, the four graphs tell a consistent story. Graph 1 establishes that delay is the norm, with the centre of the distribution at 55 minutes. Graph 2 identifies distance as the strongest single driver, with a tipping point near 10 kilometres. Graph 3 shows that delays are most numerous when order volume is highest and that the delay rate is high in every period. Graph 4 shows that traffic shifts the entire distribution to the right. No single factor explains the whole picture, and the factors reinforce each other: a long-distance order during a busy evening in heavy traffic is almost guaranteed to be late.

9.8 Summary Table of Findings

Factor	Evidence	Effect on Delay
Distance	Scatter plot, strong positive trend	Very strong; tipping point near 10 km
Traffic level	Box plot, median 51 to 65 minutes	Strong; High traffic raises median by 14 minutes
Time of day	Bar chart, volume 280 to 340 in busy periods	Moderate; busy periods hold most delayed orders
Overall performance	Histogram, peak at 48 to 55 minutes	System-wide; 66.4 percent exceed SLA

9.9 Limitations of the Analysis

The analysis is descriptive and does not prove causation. The dataset has only 1,000 records and may not represent every city or season. Simple imputation can slightly distort distributions. The 45-minute SLA is an assumed benchmark. Weather, vehicle type, preparation time, and courier experience were not visualized in the four core charts and deserve separate analysis. Finally, interactions between variables, such as distance combined with traffic, were not modelled formally.

10. Future Scope:

10.1 Real-Time GPS and IoT Integration

The current analysis is retrospective. A major extension is to connect the system to live data streams. Courier smartphones and vehicle-mounted IoT devices can transmit GPS coordinates, speed, and stop durations every few seconds. Such telemetry would replace the static Distance_km column with the actual road distance travelled and would reveal where couriers lose time, such as at traffic signals, restaurant waiting areas, or building entrances. Live traffic feeds and weather APIs could be merged with the order stream so that Traffic_Level and Weather are measured objectively instead of categorized manually. Streaming frameworks and dashboards could present the delay proportion in real time, allowing dispatchers to react within minutes instead of days. IoT-enabled insulated delivery bags could also monitor food temperature, creating a new quality KPI beyond time.

10.2 Machine Learning Regression Model for ETA Prediction

The next logical step is to build a predictive model. Using scikit-learn, the dataset can be split into training and test sets, and models such as multiple linear regression, random forest regression, and gradient boosting can be trained to predict Delivery_Time_min from Distance_km, Weather, Traffic_Level, Time_of_Day, Vehicle_Type, Preparation_Time_min, and Courier_Experience_yrs. Categorical variables would be encoded using one-hot encoding. Model quality would be assessed using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R-squared). Feature-importance analysis would then quantify which factors matter most. A classification model for Is_Delayed, evaluated using accuracy, precision, recall, and F1-score, could provide an early warning that an order is likely to breach the SLA so that the platform can reassign it or notify the customer in advance.

10.3 Dynamic Surge Pricing

Predictive and descriptive insights can feed a pricing strategy. When the model forecasts high congestion and heavy demand, the platform could introduce a temporary surge fee or courier incentive, encouraging more couriers to log in and moderating demand. Distance-based pricing could be applied so that long-distance orders, which are the most likely to be late and the most costly to serve, carry an appropriate fee. Such pricing must be designed fairly and transparently to avoid customer backlash.

10.4 Geofenced Delivery Caps

Because the scatter plot shows that orders beyond about 10 kilometres are almost always delayed, the platform could introduce geofencing, a virtual boundary around each restaurant or dark kitchen. Orders originating outside the boundary would either be declined, be offered with a longer ETA, or be fulfilled by a closer kitchen. Dynamic geofences could shrink during peak hours and expand during quiet periods so that service reliability is maintained.

10.5 Additional Extensions

Route optimization and order batching. Algorithms from the vehicle routing literature could be used to batch nearby orders and sequence them efficiently.

Courier-matching intelligence. Courier experience and vehicle type could be used to match difficult orders to suitable couriers.

Kitchen-side optimization. Preparation time can be predicted and used to time courier arrival so that couriers do not wait.

Explainable AI. Techniques such as SHAP values would help managers to trust and understand model predictions.

Deployment as a web application. A Flask or Streamlit interface could allow managers to upload new data and instantly view updated dashboards.

Larger and richer datasets. Collecting data over multiple cities and seasons would improve generalization.

Workforce welfare analytics. Monitoring hours and delay-related stress could support fairer scheduling and safety policies.

11. Conclusion:

This project analysed a dataset of 1,000 food delivery orders to understand why deliveries frequently exceed the 45-minute SLA. After cleaning the data through mode imputation for Weather, Traffic_Level, and Time_of_Day and median imputation for Courier_Experience_yrs, and after creating the Is_Delayed feature, the team computed key indicators: a mean delivery time of 56.73 minutes, a median of 55.50 minutes, a minimum of 8 minutes, a maximum of 153 minutes, and a delay proportion of 66.4 percent. These figures reveal that late delivery is the typical outcome in the analysed data rather than an exceptional event.

The four visual analyses identified the principal delay drivers. **Distance** is the strongest factor, with delays becoming almost certain beyond roughly 10 kilometres and variability increasing sharply with distance. **Traffic** shifts the entire delivery-time distribution upward, with the median rising from 51 minutes in Low traffic to 65 minutes in High traffic. **Time of day** matters mainly through volume: the Morning, Afternoon, and Evening periods each handle roughly 280 to 340 orders and hold the largest numbers of delayed orders, while Night is lighter and slightly better in delay rate. Finally, the **overall distribution** shows that the centre of performance lies near 50 to 55 minutes, about 10 minutes beyond the benchmark.

Strategic recommendations. On the basis of these findings, the project recommends (1) publishing realistic, distance-aware ETAs or revising the SLA; (2) restricting or repricing deliveries beyond about 10 kilometres through geofencing and distance-based fees; (3) strengthening courier capacity and incentives during the Morning, Afternoon, and Evening periods; (4) integrating live traffic and weather information into dispatching and ETA quotation; (5) investigating the extreme outliers above 100 minutes through root-cause review; and (6) developing predictive machine-learning models and real-time dashboards as described in the Future Scope.

From a learning perspective, the project allowed the group to apply Python, Pandas, NumPy, Matplotlib, and Seaborn to a realistic logistics problem, to practice responsible handling of missing data, to design informative visualizations, and to translate statistical findings into managerial decisions. The work demonstrates that even a modest dataset, when analysed carefully, can reveal actionable insights about service quality.

In conclusion, improving food delivery performance requires a combined strategy of shorter service radii, smarter peak-hour capacity planning, and traffic-aware operations, supported by continuous data analytics. The analysis in this report provides a clear evidence base and a reproducible foundation for these improvements.

12. References (IEEE Style):

[1] G. B. Dantzig and J. H. Ramser, "The truck dispatching problem," *Management Science*, vol. 6, no. 1, pp. 80-91, 1959.

[2] P. Toth and D. Vigo, Eds., *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed. Philadelphia, PA, USA: SIAM, 2014.

[3] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269-271, 1959.

[4] D. Reyes, A. Erera, M. Savelsbergh, S. Sahasrabudhe, and R. O'Neil, "The meal delivery routing problem," *Optimization Online*, 2018.

[5] Z. Wang, K. Fu, and J. Ye, "Learning to estimate the travel time," in *Proc. 24th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, London, U.K., 2018, pp. 858-866.

[6] Y. Li, K. Fu, Z. Wang, C. Shahabi, J. Ye, and Y. Liu, "Multi-task representation learning for travel time estimation," in *Proc. 24th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, London, U.K., 2018, pp. 1695-1704.

[7] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.

[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, 2016, pp. 785-794.

[9] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[10] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA, USA: Addison-Wesley, 1977.

[11] B. W. Silverman, *Density Estimation for Statistics and Data Analysis*. London, U.K.: Chapman and Hall, 1986.

[12] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.

[13] R. J. A. Little and D. B. Rubin, *Statistical Analysis with Missing Data*, 3rd ed. Hoboken, NJ, USA: Wiley, 2019.

[14] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf. (SciPy)*, Austin, TX, USA, 2010, pp. 56-61.

[15] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357-362, 2020.

[16] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science and Engineering*, vol. 9, no. 3, pp. 90-95, 2007.

[17] M. L. Waskom, "seaborn: statistical data visualization," *Journal of Open Source Software*, vol. 6, no. 60, p. 3021, 2021.

[18] "Food Delivery Times Dataset," Kaggle. [Online]. Available: https://www.kaggle.com (accessed 2026).